Comparative Performance Analysis of Sequential C, Auto-Vectorized C, and AVX2 Assembly implementation of Sobel Edge Detection Across Different Image Resolutions

---

Group 1's Members:

- Catapang, Nathan James
- Garzon, Harris Martin
- Ruiz, Joseph Benjamin
- Tamayo, Chelsea Nichole

I. Abstract

Image filtering is a common operation in image processing and within the field of technology. Yet, it is not limited to just technology. Medical and industrial fields utilize this feature to reduce noise within images, clear and smooth out surroundings, and enhance visuals to perform optimally. However, applying filters to high-resolution images can involve substantial computational work due to the high amount of pixels needed to be processed. This process is usually done one at a time through sequential implementation, increasing the processing time as image resolution grows. This project aims to investigate the use of data-level parallelism to improve the performance of image filtering. The project will implement Sobel edge detection using three approaches: Sequential C, compiler-auto-vectorized C, and x86-64 AVX2 assembly. The performance of these implementations will be evaluated using images with different resolutions, with execution time and speedup serving as the primary metrics. The output images will also be compared pixel by pixel to check whether all three implementations produce the correct results.

II. Proposed Project Description

A. Inputs

The inputs will be images in PNG and JPEG formats. These images will be converted into grayscale before being processed. This is because the Sobel Edge detection algorithm detects edges by calculating changes in pixel intensity between neighboring pixels. Converting the images first will allow the algorithm to work with only one intensity value per pixel instead of processing the red, green, and blue values separately. This will make the calculations simpler and reduce the amount of data that needs to be processed. The project will also use images with different dimensions to compare the execution times of the sequential C, compiler-auto-vectorized C, and x86-64 AVX2 assembly implementations and see how image size affects their performance.

Each pixel will be represented by an 8-bit unsigned integer with an intensity value ranging from 0 to 255, where 0 represents black and 255 represents white. The pixel values will be stored in a one-dimensional array. The image width and height will be used to determine the position of each pixel so that all three implementations will use the same input data when performing the Sobel calculations. Using the same images will also help ensure a fair comparison of their execution

times and allow the project to check whether all three implementations produce the correct output.

B. Proposed Process

The project will focus on implementing the Sobel edge detection algorithm using three approaches: Sequential C, compiler-auto-vectorized C, and x86-64 AVX2 assembly. The project aims to identify and compare the performance of these implementations across different image resolutions and determine how their execution times and relative speedups change as the amount of image data increases.

Firstly, the input image will be converted into grayscale to simplify the process by representing each pixel using a single intensity value instead of separate red, green and blue values. This helps the Sobel algorithm detect changes in pixel intensity and makes the edge detection process more straightforward. Afterwards, the algorithm will calculate the horizontal G_x and vertical G_y intensity gradients of each pixel using the two 3x3 convolution kernels. These gradients will be combined using the gradient magnitude formula, $G = \sqrt{G_x^2 + G_y^2}$, to determine the strength of the edges in the image. Pixels with larger gradient magnitudes will indicate more prominent edges, which will be highlighted in the resulting output image. The input grayscale pixels will use 8-bit unsigned integers, with values ranging from 0 to 255. The horizontal G_x and vertical G_y gradients will use 16-bit signed integers since their values can range from -1020 to 1020. The squared gradients and their sum will use 32-bit integers to prevent overflow during the calculation. The gradient magnitude will be rounded to the nearest integer, limited to the range of 0 to 255, and converted into an 8-bit unsigned integer before being stored in the output image.

To evaluate the benefits of data-level parallelism, the sequential implementation will serve as the baseline, processing the image pixel by pixel with compiler auto-vectorization disabled. Secondly, the compiler-auto-vectorized C version will use optimized C code and allow the compiler to generate SIMD instructions. The generated SIMD instructions will be verified using compiler optimization reports or generated assembly. Lastly, the AVX2 implementation will process multiple pixels simultaneously using x86-64 assembly instructions. AVX2 provides 256-bit vector registers, allowing eight 32-bit integer values or sixteen 16-bit integer values within a single vector register. The implementation will load neighboring pixels in groups, arrange the pixel values needed for the 3x3 Sobel neighborhoods, and apply the convolution coefficients using packed integer operations. The horizontal and vertical gradients will be combined using the gradient magnitude formula. The resulting intensities will then be limited to the range of 0 to 255 and packed into 8-bit output pixels.

The project will also explicitly handle image borders to prevent invalid memory accesses. Since the Sobel operator requires a complete 3x3 neighborhood, the outermost row and column will be assigned an output intensity of zero for both implementations. The convolution will only be applied to the interior pixels. For the compiler-autovectorized C and AVX2 implementations, a scalar cleanup loop will process any remaining interior pixels that do not fit into a complete

SIMD group. All three implementations will also use the same border-handling procedure to ensure that the output comparison is consistent.

Research done by Jain et al. (2016), who investigated parallel Sobel edge detection using CUDA C and compared its performance with sequential execution. Although their implementation uses GPU-based parallelism, this project will explore CPU-based SIMD parallelism using compiler auto-vectorization and AVX2. Their research provides reference for investigating parallel Sobel processing but does not establish the performance improvement that the proposed implementations will achieve. Furthermore, a study done by Almomany et al. (2020) evaluates Sobel implementations using different image sizes but focuses primarily on FPGA-based acceleration and CPU comparisons. In contrast, Peng-O and Chaikan (2021), investigates software optimizations using C, AVX intrinsics, and OpenMP to improve processing time and energy efficiency. The studies provide a foundation for examining resolution-dependent performance and SIMD optimization. However, these studies primarily address resolution based performance and implementation based optimization separately. This study aims to address both aspects by comparing all implementations across multiple image resolutions.

All three implementations will use the same input images at several resolutions to ensure fair comparison. Their execution times will be measured across different image resolutions. Speedup will be calculated by dividing the sequential C execution time by the parallel execution time across different image resolutions to evaluate performance. The comparison will examine whether compiler auto-vectorization and manually implemented AVX2 instructions improve execution time and whether the observed speedups change with image resolution. To verify correctness, the output images produced by all three implementations will be compared pixel by pixel to check whether they produce matching intensity values.

C. Outputs

The main outputs of the project will be the edge-detected images saved to disk in the target output format, along with detailed execution measurements. These outputs will provide performance benchmarks comparing sequential execution, compiler-auto-vectorized execution, and AVX2-accelerated execution across various image resolutions.

The results will also include a comparison of the execution times and speedups of the compiler-auto-vectorized C and AVX2 implementations against the sequential C implementation. The outputs of all three implementations will be compared pixel by pixel to check whether they produce matching edge-detected images. The execution times and speedups will be presented in tables to make it easier to compare their performance across different image resolutions.

III. Target Platform

The project will use C for the sequential implementation and compiler auto-vectorization, and x86-64 AVX2 assembly language for the AVX2 implementation. The assembly kernel will be called from the C program. Development and testing will be performed on a computer that supports AVX2. GCC and NASM will be used to compile and execute the program. The computer used for testing must support

AVX2 instructions to execute the AVX2 implementation. Compiler optimization reports or generated assembly will be used to verify that GCC generates SIMD instructions for the compiler-auto-vectorized C implementation.

Performance will be evaluated by comparing the execution times of all three implementations across different image resolutions. The speedup of the compiler-auto-vectorized C and AVX2 implementations will also be calculated relative to the sequential C implementation.

IV. Mid-Project Milestone

By week 10, the sequential C implementation, compiler-auto-vectorized C implementation, and an initial version of the AVX2 assembly implementation of the Sobel edge detection algorithm should be completed. All three implementations will be tested using sample images to verify that they produce correct and comparable edge-detected outputs. Preliminary execution-time measurements will also be collected using images with different resolutions to assess the performance of the compiler-auto-vectorized C and AVX2 implementations compared to the sequential baseline. The output images will also be compared pixel by pixel to check for correctness. Any errors or performance issues identified during testing will be addressed before the final demonstration.

V. References

Almomany, A., Al-Omari, A., Jarrah, A., Tawalbeh, M., & Alqudah, A. (2020). An OpenCL-based parallel acceleration of aSobel edge detection algorithm Using IntelFPGA technology. *South African Computer Journal*, *32*(1). https://doi.org/10.18489/sacj.v32i1.749

Jain, A., Namdev, A., & Chawla, M. (2016). Parallel edge detection by SOBEL algorithm using CUDA C. *IEEE Students Conference on Electrical, Electronics and Computer Science*, 1–6. https://doi.org/10.1109/sceecs.2016.7509360

Moukthika. (2025, May 6). Edge detection using OpenCV. OpenCV. https://opencv.org/edge-detection-using-opencv/

Peng-O, T., & Chaikan, P. (2021). High performance and energy efficient sobel edge detection. *Microprocessors and Microsystems*, *87*, 104368. https://doi.org/10.1016/j.micpro.2021.104368